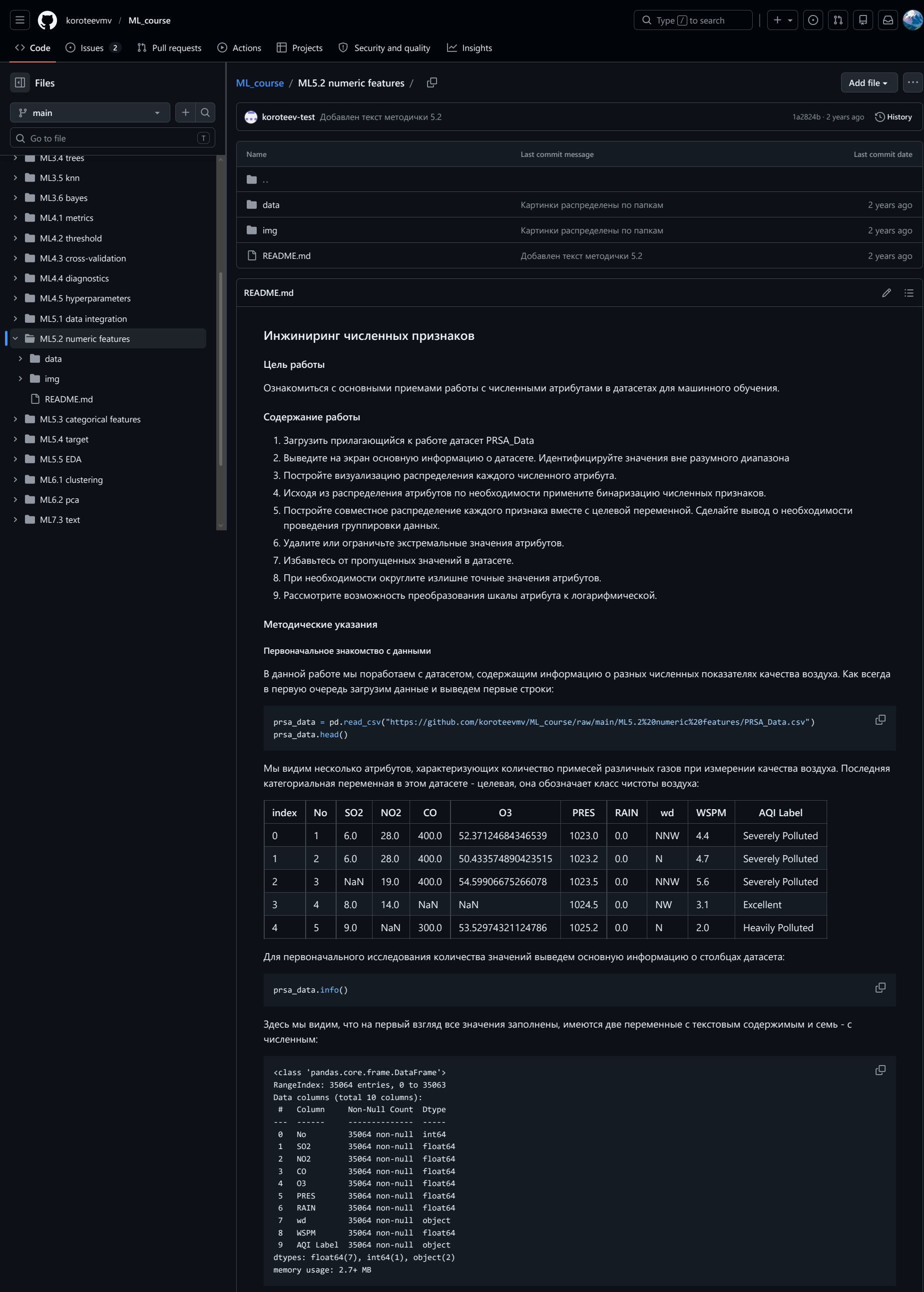


Инжиниринг численных признаков

Цель работы

Ознакомиться с основными приемами работы с численными атрибутами в датасетах для машинного обучения.

Содержание работы

1. Загрузить прилагающийся к работе датасет PRSA_Data
2. Выведите на экран основную информацию о датасете. Идентифицируйте значения вне разумного диапазона
3. Постройте визуализацию распределения каждого численного атрибута.
4. Исходя из распределения атрибутов по необходимости примените бинаризацию численных признаков.
5. Постройте совместное распределение каждого признака вместе с целевой переменной. Сделайте вывод о необходимости проведения группировки данных.
6. Удалите или ограничьте экстремальные значения атрибутов.
7. Избавьтесь от пропущенных значений в датасете.
8. При необходимости округлите излишне точные значения атрибутов.
9. Рассмотрите возможность преобразования шкалы атрибута к логарифмической.

Методические указания

Первоначальное знакомство с данными

В данной работе мы поработаем с датасетом, содержащим информацию о разных численных показателях качества воздуха. Как всегда в первую очередь загрузим данные и выведем первые строки:

```
prsa_data = pd.read_csv("https://github.com/koroteevmv/ML_course/raw/main/ML5.2%20numeric%20features/PRSA_Data.csv")
prsa_data.head()
```

Мы видим несколько атрибутов, характеризующих количество примесей различных газов при измерении качества воздуха. Последняя категориальная переменная в этом датасете - целевая, она обозначает класс чистоты воздуха:

index	No	SO2	NO2	CO	O3	PRES	RAIN	wd	WSPM	AQI Label
0	1	6.0	28.0	400.0	52.37124684346539	1023.0	0.0	NNW	4.4	Severely Polluted
1	2	6.0	28.0	400.0	50.433574890423515	1023.2	0.0	N	4.7	Severely Polluted
2	3	NaN	19.0	400.0	54.59906675266078	1023.5	0.0	NNW	5.6	Severely Polluted
3	4	8.0	14.0	NaN	NaN	1024.5	0.0	NW	3.1	Excellent
4	5	9.0	NaN	300.0	53.52974321124786	1025.2	0.0	N	2.0	Heavily Polluted

Для первоначального исследования количества значений выведем основную информацию о столбцах датасета:

```
prsa_data.info()
```

Здесь мы видим, что на первый взгляд все значения заполнены, имеются две переменные с текстовым содержимым и семь - с численным:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 35064 entries, 0 to 35063
Data columns (total 10 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   No         35064 non-null  int64  
 1   SO2        35064 non-null  float64
 2   NO2        35064 non-null  float64
 3   CO         35064 non-null  float64
 4   O3         35064 non-null  float64
 5   PRES       35064 non-null  float64
 6   RAIN       35064 non-null  float64
 7   wd         35064 non-null  object 
 8   WSPM       35064 non-null  float64
 9   AQI Label  35064 non-null  object 
dtypes: float64(7), int64(1), object(2)
memory usage: 2.7+ MB
```

Так как большая часть атрибутов датасета выражена численно, информативным будет посмотреть основную численную статистику по данным:

```
prsa_data.describe()
```

При внимательном рассмотрении нас должно смутить, что минимальное значение по многим столбцам, означающим концентрацию определенных газов, является значение -1:

index	No	SO2	NO2	CO	O3	
count	35064.0	35064.0	35064.0	35064.0	35064.0	35064.0
mean	17532.5	18.058733698380106	63.236860663358435	1251.1216917636323	45.876245676104766	1010.52
std	10122.24925597073	22.558126329870454	39.10923159613662	1269.0335562739235	54.72944847600266	26.2892
min	1.0	-1.0	-1.0	-1.0	-1.0	-1.0
25%	8766.75	4.0	34.0	500.0	2.400026434533582	1002.5
50%	17532.5	10.0	58.0	900.0	27.37830570188505	1010.8
75%	26298.25	22.2768	87.0	1500.0	69.57736619098368	1019.4
max	35064.0	282.0	264.0	10000.0	364.3447496563047	1040.3

Исследование других статистик не дает нам пока никакой значимой очевидной информации, за исключением того, что разные столбцы имеют разные максимальные значения, то есть они измеряются по разным количественным шкалам. Это может навести нас на мысль о необходимости использования нормализации.

Идентификация ошибочных значений

Первым этапом анализа данных должно быть устранение явных ошибок в данных - значений атрибутов вне разумного диапазона. На предыдущем этапе мы уже обнаружили эту проблему. Значение концентрации газа в воздухе просто по смыслу не может быть отрицательным. При этом во всем датасете во всех колонках присутствует только одно отрицательное значение (убедитесь в этом самостоятельно): -1. Скорее всего, так в датасете обозначаются неизвестные значения.

Такие "специальные значения" в численных атрибутах надо исправлять. Заменим их на np.nan, чтобы они явно отображались как пропуски и мы смогли в дальнейшем корректно с ними работать:

```
prsa_data[prsa_data == -1] = np.nan
prsa_data.head()
```

Для проверки корректности замены выведем шапку датасета. Убедимся, что в данных появились пропуски:

index	No	SO2	NO2	CO	O3	PRES	RAIN	wd	WSPM	AQI Label
0	1	6.0	28.0	400.0	52.37124684346539	1023.0	0.0	NNW	4.4	Severely Polluted
1	2	6.0	28.0	400.0	50.433574890423515	1023.2	0.0	N	4.7	Severely Polluted
2	3	NaN	19.0	400.0	54.59906675266078	1023.5	0.0	NNW	5.6	Severely Polluted
3	4	8.0	14.0	NaN	NaN	1024.5	0.0	NW	3.1	Excellent
4	5	9.0	NaN	300.0	53.52974321124786	1025.2	0.0	N	2.0	Heavily Polluted

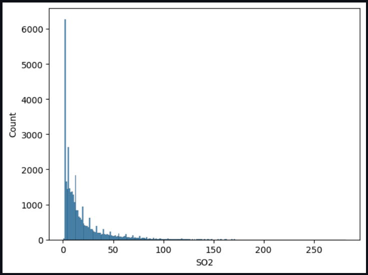
Самостоятельно повторите вывод статистики и убедитесь, что все численные столбцы принимаю неотрицательные значения.

Визуализация распределения атрибутов

После того, как мы вычистили явные ошибки в данных, можно познакомиться с эмпирическим распределением каждого признака. Для этого можно использовать разные средства визуализации, например, гистограмму:

```
sns.histplot(prsa_data.SO2)
```

Здесь мы видим, какие значения концентрации SO2 присутствуют в датасете:

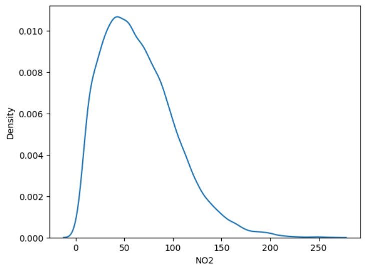


Налицо явная зависимость - чем меньше значение, тем оно вероятнее. Очень большие концентрации встречаются крайне редко. Плюс, на графике можно заметить определенные пики через равные промежутки значений. Это может свидетельствовать о попытках заполнения пропущенных значений или об округлении некоторых значений.

Кроме гистограммы можно построить, например, график ядерной оценки плотности распределения:

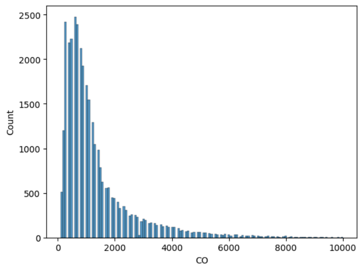
```
sns.kdeplot(prsa_data.NO2)
```

Но обратите внимание, что оценка плотности подразумевает некоторое усреднение. Поэтому не удивляйтесь тому, что график немного заходит на отрицательную область, хотя мы специально устраняли это не предыдущем этапе:



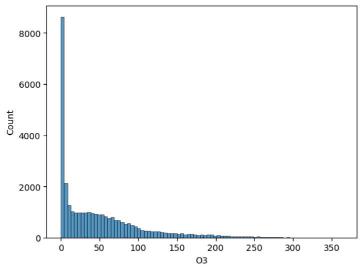
Здесь мы видим уже другое распределение - более распространены значения, близкие к 50.

В дальнейшем будем пользоваться гистограммами. Вот распределение значений концентрации CO:

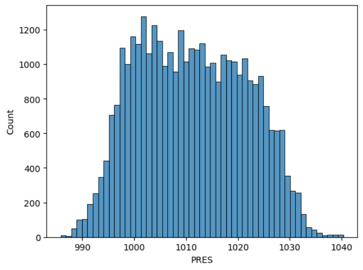


Здесь видно достаточно гладкое распределение с модой в районе 1000.

Вот более смещенное распределение - O3. На гистограмме явно видно, что нулевая концентрация встречается гораздо чаще ненулевых значений. В дальнейшем это может быть кандидат на бинаризацию:

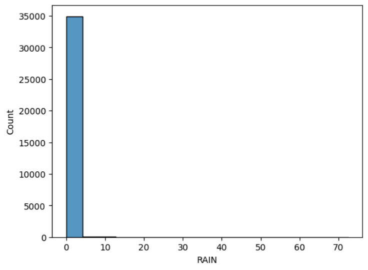


Распределение значения давления отличается от всех предыдущих. Видно, что есть некоторый диапазон условно равновероятных, "нормальных" значений, а частота значений давления вне этого диапазона быстро спадает:

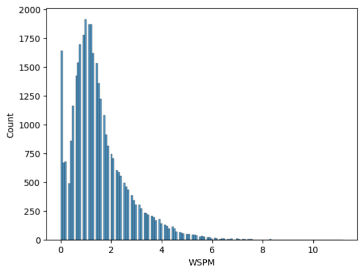


Такие атрибуты тоже можно категоризовывать при необходимости.

Распределение количества осадков тоже особенное:



Мы видим, что отсутствие осадков - самое распространенное значение. Ненулевые значения тут встречаются в подавляющем меньшинстве точек данных:



Вот с обработки этого атрибута и начнем преобразование данных.

Бинаризация атрибутов

Такие распределения, как в атрибуте "количество осадков" могут повредить эффективности модели, так как относительное численной значение при таком распределении не несет, по сути, статистической значимости по сравнению с информацией о том, были ли вообще осадки или нет. То есть этот атрибут из численного можно превратить в категориальный. Ведь в нашем датасете всего 1436 точек из 35 000 имеют ненулевое значение:

```
prsa_data.RAIN[prsa_data.RAIN > 0]
```

```
267      0.1
268      0.4
269      0.1
270      0.9
271      0.9
        ...
34891    0.2
34892    0.7
34893    0.9
34894    0.4
34895    0.2
Name: RAIN, Length: 1436, dtype: float64
```

Так важно ли будет для модели конкретное численное значение количества осадков при таком малом относительном объеме? Мы можем ввести в модель новый категориальный, а точнее, бинарный атрибут, кодирующий наличие либо отсутствие осадков в данном измерении:

```
is_rain = np.array(prsa_data.RAIN)
is_rain[is_rain > 0] = 1
prsa_data['IS_RAIN'] = is_rain
prsa_data.drop(['RAIN'], axis=1, inplace=True)
prsa_data.describe()
```

Обратите внимание, что для воспроизводимости кода предпочтительнее такие преобразования не производить над существующей колонкой, а создавать новую. Старую всегда можно будет удалить. Либо исследовать относительную важности признаков и принять решение после этого. А вот как будет выглядеть датасет после преобразования:

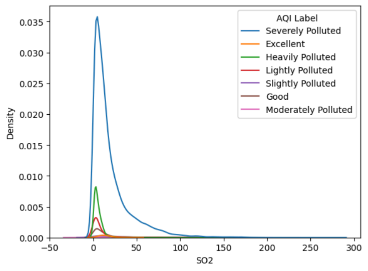
index	No	SO2	NO2	CO	O3	
count	35064.0	34489.0	33994.0	33252.0	32957.0	35044.
mean	17532.5	18.376480570616717	65.25878926575278	1319.3535125706724	48.8731279663482	1011.0
std	10122.24925597073	22.609647740471033	37.99608792528771	1268.1143306714334	55.11211847974543	10.355
min	1.0	0.2856	1.6424	100.0	-0.24025790735349623	985.9
25%	8766.75	4.0	36.0	500.0	3.5767910527049533	1002.5
50%	17532.5	10.0	60.0	900.0	31.992632810880806	1010.8
75%	26298.25	23.0	88.0	1600.0	72.89152376395555	1019.4
max	35064.0	282.0	264.0	10000.0	364.3447496563047	1040.3

Визуализация связи атрибутов с целевой переменной

После первоначального преобразования данных нужно определить, как именно связаны различные атрибуты со значением целевой переменной. Так как мы исследуем численные атрибуты при категориальной целевой переменной, то воспользуемся визуализацией нескольких оценок плотности распределения для разных классов:

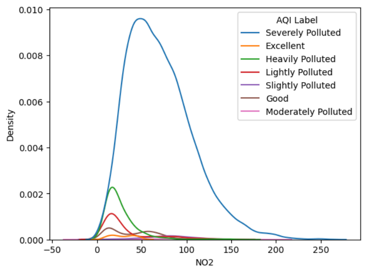
```
sns.kdeplot(data=prsa_data, x="SO2", hue="AQI Label")
```

Мы получим на графике несколько линий, каждая из которых соответствует распределению выбранного атрибута при разных значениях целевой переменной:

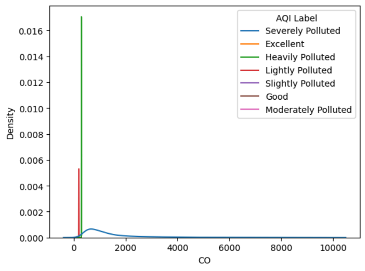


Так как у нас наблюдается дисбаланс классов (это мы еще не исследовали, но из данного графика ясно видно, что всего объектов разных классов сильно разное количество), то нам не очень важен на таких графиках конкретный уровень каждой линии. Важно понять, есть ли участки на горизонтальной оси, на которых соотношение разных существенно отличается.

Например, на первом графике, явных изменений в форме разных линий нет. Можно предположить, что изолированно данный атрибут не сильно влияет на значение целевой переменной. Пойдем дальше и визуализируем второй атрибут:



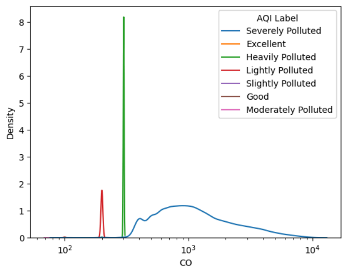
Здесь уже видны различия между соотношениям разных классов. Но они не очень ярко выраженные. Перейдем к третьему атрибуту:



Данный график очень смещен из-за того, что плотности некоторых классов сильно сконцентрированы, что искажает масштаб графика. В таких случаях помогает попробовать изобразить этот же график, но с логарифмическим масштабом вертикальной оси:

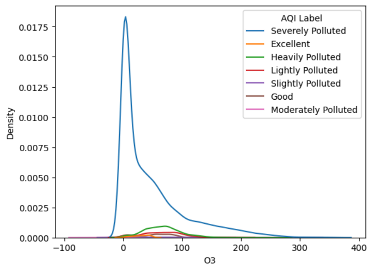
```
sns.kdeplot(data=prsa_data, x="CO", hue="AQI Label", log_scale=True)
```

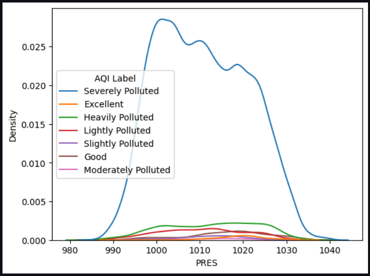
Теперь мы четко видим те самые различия:

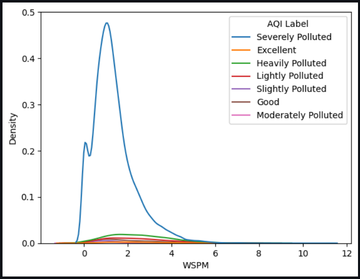


На некоторых участках диапазона значений данного атрибута преобладает значение "Lightly polluted", на других - "Heavily polluted". Это значит, что данный атрибут очень важен для предсказания значения целевой переменной. В дальнейшем мы используем эту информацию для категоризации этого атрибута.

Следующие графики демонстрируют схожее поведение разных классов с поправкой на общую форму распределения разных атрибутов:



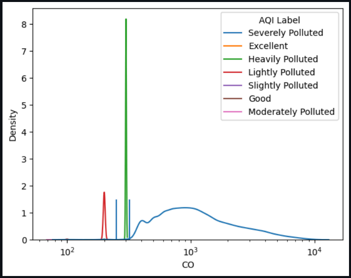




В целом, анализ влияния атрибутов показывает, что их влияние скорее проявляется в совокупности. Нет одного четко дифференцирующего фактора, особенно указывающего на все возможные значения целевой переменной.

Группировка численных значений

В некоторых случаях бывает полезно объединить численные значения атрибутов на метку диапазона значений. То есть мы объединяем (группируем) объекты по значению данного атрибута и затем заменяем сам численный атрибут на ту группу, в которую попал объект. Этот прием называется дискретизация или группировка численных значений. Например, возьмем атрибут "CO". На предыдущем шаге мы уже выяснили, что объекты, находящиеся в различных диапазонах его значений будут иметь разное распределение целевой переменной. Условные границы этих диапазонов можно изобразить на графике:



В данном случае, для значения целевой переменной не так важно конкретное численной значение концентрации CO в воздухе. Достаточно информации, в какой диапазон попадает данное измерение. Поэтому попробуем добавить к датасету атрибут "Группа по CO":

```
bin_ranges = [0, 250, 320, 10000]
bin_names = [1, 2, 3]
prsa_data['CO_bin_custom_label'] = pd.cut(np.array(prsa_data['CO']),
                                          bins=bin_ranges, labels=bin_names)
prsa_data.head()
```

В данном примере, границы атрибутов мы определяли вручную, по графику выше. Можно их определять автоматически, через изменение долей распределения целевой переменной, либо брать в качестве границ квартили или процентили.

Здесь у нас возникает проблема. Так как в исходном столбце были пропущенные значения, в получившемся столбце тоже будут пропуски. Так как этот новый атрибут - категориальный, проще всего сразу заполнить пропущенные значения специальным названием класса. В данном случае, естественно будет присвоить этому классу метко "0", что будет означать - "значение неизвестно":

```
prsa_data['CO_bin_custom_label'] = prsa_data['CO_bin_custom_label'].values.add_categories(0)
prsa_data['CO_bin_custom_label'] = prsa_data['CO_bin_custom_label'].fillna(0).astype(int)
prsa_data.head()
```

Вот как выглядит наш датасет после преобразования:

index	No	SO2	NO2	CO	O3	PRES	wd	WSPM	AQI Label	IS_RAIN	CO_bin_custom_lab
0	1	6.0	28.0	400.0	51.66378173181024	1023.0	NNW	4.4	Severely Polluted	0.0	3
1	2	6.0	28.0	400.0	49.72659979230651	1023.2	N	4.7	Severely Polluted	0.0	3
2	3	NaN	19.0	400.0	55.270067369938324	1023.5	NNW	5.6	Severely Polluted	0.0	3
3	4	8.0	14.0	NaN	NaN	1024.5	NW	3.1	Excellent	0.0	0
4	5	9.0	NaN	300.0	54.22212476907075	1025.2	N	2.0	Heavily Polluted	0.0	2

Обратите внимание, что мы можем не удалять сразу исходный столбец. Обычно все ненужные столбцы удаляются из датасета в самом конце, перед массовым преобразованием категориальных переменных. Либо решение о целесообразности удаления можно принято позже, по результатам дальнейшего анализа.

Удаление экстремальных значений

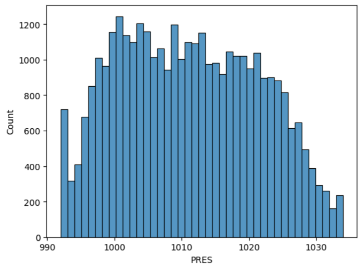
При анализе распределений особый интерес вызывают такие, у которых наблюдаются длинные "хвосты" - то есть очень редкие значения далеко от среднего или медианного значения. Это такие объекты, которых очень мало (может быть, даже один) и которые по значению данной характеристики сильно отличаются от основной массы объектов. Часто такие выбросы свидетельствуют о некорректных данных, либо о аномальных и непоказательных объектах. Объекты, которые нерелевантны решаемой задаче следует удалять из выборки. Но сами по себе длинные "хвосты" распределений могут повредить работе моделей. Поэтому зачастую датасеты с большими выбросами очищают от них автоматически.

Для этого можно удалить объекты, значения определенного атрибута у которых выше или ниже определенного порога или процентиля. Но в таком случае мы лишаемся определенного количества информации, которая была заключена в значении других атрибутов данного объекта. Альтернативный способ - клиппинг. Он заключается в том, что мы заменяем экстремальные значения атрибута граничным.

Возьмем для примера колонку "PRES" нашего датасета. В данном примере определим границы экстремальных значений опять же на глаз. И заменим все значения, находящиеся вне этих границ:

```
prsa_data.PRES[prsa_data.PRES <= 992] = 992
prsa_data.PRES[prsa_data.PRES >= 1034] = 1034
```

Посмотрите, как это изменяет форму эмпирического распределения данного атрибута:



Поэкспериментируйте с клиппингом с разными значениями границ. Посмотрите, как они влияют на форму получившегося распределения. Поробуйте применить клиппинг к другим атрибутам этого датасета.

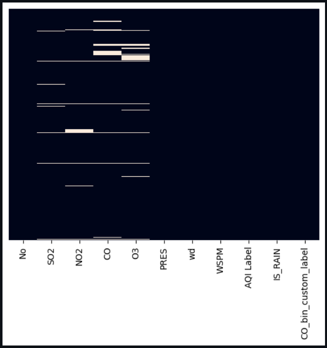
Заполнение пропусков

Обязательной частью предварительной обработки данных является устранение отсутствующих значений. В первую очередь нужно позаботиться о том, чтобы все реально имеющиеся в данных пропуски отображались в датасете как np.nan или другое детектируемое значение. Это мы уже сделали.

Теперь можно визуализировать пропущенные значения в данных:

```
sns.heatmap(prsa_data.isnull(), yticklabels=False, cbar=False)
```

Данная диаграмма показывает графически как расположены пропуски в данных:



Можно обратить внимание на то, что в датасете прослеживаются горизонтальные полосы. Это объекты, по которым неизвестны значения нескольких атрибутов. Можно вывести индексы объектов, у которых неизвестно значение более двух атрибутов:

```
undef = prsa_data.isnull().sum(axis=1)
undef[undef >= 2]
```

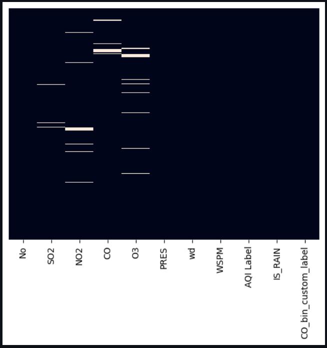
Здесь мы видим, что таких объектов всего 884:

```
3        2
276      3
435      2
459      2
555      2
        ..
34880    4
34883    4
34885    4
35029    2
35030    2
Length: 884, dtype: int64
```

Так как такие незаполненные объекты составляют очень малый процент выборки, для простоты обработки их можно просто удалить:

```
prsa_data = prsa_data.drop(undef[undef >= 2].index, axis=0)
```

Повторим визуализацию пропусков. Теперь мы видим, что пропуски группируются по столбцам:



Выведем количество пропущенных значений для каждого атрибута:

```
prsa_data.isnull().sum()
```

Мы видим, что пропуски присутствуют в четырех столбцах. При этом в одном из них пропусков сильно меньше:

```
No                        0
SO2                     281
NO2                     702
CO                     1028
O3                     1300
PRES                      0
wd                        0
```

```
WSPM                     0
AQI Label                0
IS_RAIN                  0
CO_bin_custom_label      0
dtype: int64
```

По столбцу SO2 пропущено всего 281 значение. Это менее одного процента выборки. Можно просто заменить пропуски на среднее значение. Но так как эмпирическое распределение этого признака имеет большую асимметрию, лучше использовать медиану:

```
prsa_data.SO2 = prsa_data.SO2.fillna(prsa_data.PRES.mean())
```

Самостоятельно постройте распределение данного признака после заполнения и сравните с распределением до. Посмотрите, насколько заметно искажение при таком способе.

С другими столбцами, например, O3 искажение будет более существенным. Поэтому воспользуемся заполнением не средним, а случайным значением. Для этого сначала сформируем временную серию, содержащую все пропуски из данного столбца:

```
filler = prsa_data.O3[prsa_data.O3.isna()]
```

Нам важно, чтобы в этой серии сохранился исходный индекс - номера строк с пропусками исходного датасета. Это мы используем при заполнении этой серии данными. В этом примере не будем генерировать случайные значения их какого-то распределения, а просто семплируем их из существующих значений:

```
filler = prsa_data.O3[~prsa_data.O3.isna()].sample(n=len(filler)).set_axis(filler.index)
```

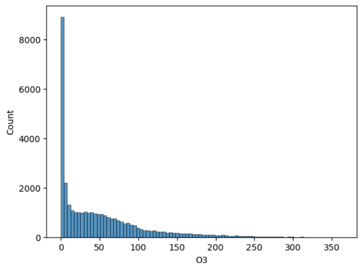
Вот как наша серия будет выглядеть:

```
436       59.726165
460        9.490829
556       52.339174
652        2.210643
748       17.802473
            ...
33124     46.354788
33220     56.869096
33604    147.957579
33892      6.986806
35031     30.261312
Name: O3, Length: 1300, dtype: float64
```

Теперь просто воспользуемся ей для заполнения пропусков в исходном датасете:

```
prsa_data.O3 = prsa_data.O3.fillna(filler)
```

Вот как выглядит распределение значений данного атрибута после заполнения пропусков:



Сравните его с распределением до заполнения.

Округление атрибутов

При визуальном осмотре датасета можно заметить, что все численные значения имеют один знак после запятой. Но столбец O3 записан с гораздо большей точностью - 5 знаков после запятой. Такая точность является избыточной, так как получается, что в этой колонке записаны 7 значащих цифр. От лишних данных можно избавиться, просто округлив данные значения:

```
prsa_data['O3'] = np.array(np.round((prsa_data['O3'])), dtype='int')
prsa_data.head()
```

index	No	SO2	NO2	CO	O3	PRES	wd	WSPM	AQI Label	IS_RAIN	CO_bin_custom_label
0	1	6.0	28.0	400.0	52	1023.0	NNW	4.4	Severely Polluted	0.0	3
1	2	6.0	28.0	400.0	50	1023.2	N	4.7	Severely Polluted	0.0	3
2	3	NaN	19.0	400.0	55	1023.5	NNW	5.6	Severely Polluted	0.0	3
4	5	9.0	NaN	300.0	54	1025.2	N	2.0	Heavily Polluted	0.0	2
5	6	8.0	17.0	300.0	54	1025.6	N	3.7	Heavily Polluted	0.0	2

На практике очень редко когда нужна точность более 2-3 значащих цифр. Но в отдельных случаях можно округлять и более грубо - по сути это является аналогом численной группировки значений.

Логарифмирование атрибутов

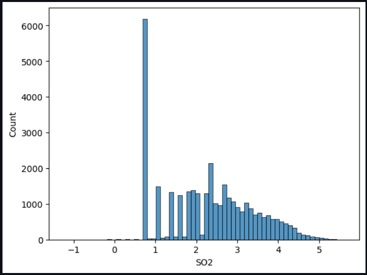
Еще один прием, который не часто применяется, но иногда бывает полезен - кастомное решкалирование значений. В целом, решкалирование - это изменение шкалы измерения атрибута. Например, изменение граничных значений, масштабирование, нормализация. Но в определенных случаях, может быть полезно применить и другое преобразование данных. Опять обратим внимание на столбец SO2. Сейчас его значения очень сильно смещены. Малых значений очень много, а значит, что даже маленькие разницы между ними могут быть существенны для модели. А вот больших значений очень мало, и даже большие расстояния между ними могут быть не очень показательны.

Вообще, математические модели лучше всего работают с распределениями, похожими на равномерное или нормальное. Очень большой дисбаланс в распределении может быть вреден для эффективности.

Мы можем попробовать применить к атрибуту любую математическую функцию, чтобы преобразовать его к более равномерному распределению. Только эта функция должна быть непрерывная и монотонная. В данном случае, нам будет полезен логарифм:

```
prsa_data.SO2 = np.log(prsa_data.SO2)
```

Посмотрите, как после логарифмирования изменилось распределение этого признака:



При таком равномерном распределении информации модели будет легче работать с этим признаком.

Задания для самостоятельного выполнения

1. При выполнении 3 и 5 заданий используйте другие методы визуализации. Найдите самый подходящий тип графика для каждого распределения.
2. При выполнении 6 задания мы явно подбирали руками границы диапазона для клиппинга. Реализуйте адаптивный клиппинг через процентили.
3. Избавьтесь от оставшихся пропусков в данных. Самостоятельно выберите метод.
4. Проведите нормализацию численных признаков. Выберите наиболее подходящий вид нормализации для каждого признака.
5. Постройте кореллограмму по всем численным столбцам датасета. Сделайте вывод о значимости признаков.
6. Визуализируйте связи между признаками. Сделайте вывод об их взаимозависимости.

Контрольные вопросы

1. Как в датасете идентифицировать численные атрибуты?
2. Какие основные виды непрерывных распределений часто встречаются на практике анализа данных?
3. Какие типы графиков можно использовать для визуализации одномерного непрерывного распределения?
4. А совместного распределения с другой непрерывной переменной? А с категориальной переменной?
5. Как выбрать метод борьбы с пропущенными значениями?

Дополнительные задания

1. В данной работе проверьте целесообразность каждого необязательного преобразования данных путем проверки, увеличивает ли данное преобразование точность модели. Проверьте на простом виде модели (логит регрессия, дерево решений или случайный лес).
2. Проанализируйте эффективность использования различных методов заполнения пропущенных значений.
3. Попробуйте придумать другие методы инжиниринга признаков в этом датасете. Оцените их эффект по какой-то модели.
4. Перед началом обработки данных разбейте датасет на тестовую и обучающую выборки. Очистите по методу из работы обучающую выборку. Повторите обработку на тестовой выборке. При этом позаботьтесь, чтобы все параметрические преобразования (клиппинг, нормализация, группировка и так далее).
5. Создайте воспроизводимый код обработки данного датасета.
6. В датасете Customer_support, который стал результатом выполнения предыдущей работы преобразуйте все даты в абсолютные признаки. Извлеките из дат значимую информацию - день недели, время дня, день месяца. Составьте значимые временные промежутки.
7. Повторите обработку численных параметров в датасете "Титаник".
8. (*) Создайте код, реализующий алгоритм очистки данных автоматически (для любого датасета).